PYTHON PROGRAMMING



21 Days Challenge

Day-19

Day-19

ERRORS HANDLING IN PYTHON WITH EXCEPTIONS AND ASSERTIONS



Abdullah Al Zubaer



Instructor,
Artificial Intelligence & Data Science
Gobeshona Learning Academy.
B.Sc. in Internet of Things and Robotics Engineering
Gazipur Digital University, Gazipur, Bangladesh

Table Of Content

- Exceptions
- Errors Handling
- Assertions

What is a Exception

- An Exception is an error that happens while a program is running.
- It interrupts the normal flow of the program.
- If not handled → program crashes.

```
print(10 / 0)
```

Python Exceptions

1.NameError

```
print(x)    # x not defined
```

2.TypeError

```
print("5" + 2)    # string + int
```

3.ValueError

```
int("abc")    # cannot convert string to int
```

4.ZeroDivisionError

```
print(5 / 0)
```

5.IndexError

```
nums = [1,2,3]
print(nums[5])
```

Python Exceptions

6.KeyError

```
data = {"name": "Rahim"}
print(data["age"])
```

6.AttributeError

```
x = 10
x.append(5)   # int has no append()
```

7.FileNotFoundError

```
open("nofile.txt")
```

9.ImportError / ModuleNotFoundError

```
import mymodule   # not installed
```

10.IndentationError

```
def hello():
print("Hi")   # wrong indent
```

What is Exception Handling?

- Exception Handling is the process of managing errors.
- Use: try, except, else, finally.
- Prevents program from crashing.

Exception Handling

- The try block tests code that may cause an error.
- The except block runs if an error occurs.

try-except

```
try:
    print(5/0)
except:
    print("Error occurred")
```

Exception Handling (Contd..)

- You can catch specific error types (NameError, ZeroDivisionError).
- This helps to handle different errors in different ways.

```
try-except (Specific Error)

try:
    print(x)
except NameError:
    print("Variable not defined")
except:
    print("Some other error")
```

Exception Handling (Contd..)

- else runs only if no error occurs in the try block.
- If there is an error → else is skipped.

```
try-except-else

try:
    print("Hello")
except:
    print("Error")
else:
    print("No error found")
```

Exception Handling (Contd..)

- finally runs no matter what happens (error or no error).
- Commonly used for cleanup (like closing files or database connections).

```
try-except-finally
try:
    print(5/0)
except:
    print("Error occurred")
finally:
    print("This block always runs")
```

Raise Error in Python

- The raise keyword in Python is used to manually trigger (raise) an exception.
- It allows developers to stop the program when a certain condition is met.
- You can raise built-in exceptions (like ValueError, TypeError, etc.) or create custom exceptions.

```
x = -1


if x < 0:
    raise Exception("Negative numbers are not allowed")
```

Assertion in Python

- An assertion is a way to test if a condition is true while the program runs.
- If the condition is True → program continues normally.
- If the condition is False → Python raises an AssertionError.
- Used mainly for debugging and validation.

```
x = -3
assert x > 0, "x must be positive"
print("x is positive")
```

Exception Handling – Summary

- Exception → Runtime error that interrupts program.
- try → Test risky code.
- except → Handle error.
- else → Runs if no error.
- finally → Runs always (cleanup).
- raise → Manually throw an error.
- assert → Debug/test condition → raises AssertionError if false.
- Common Exceptions:

NameError, TypeError, ValueError, ZeroDivisionError, IndexError, KeyError, AttributeError, FileNotFoundError, ImportError, IndentationError.

ASSIGNMENT

Write a Python program that asks the user for a number and divides 100 by it. Use try–except to handle errors like division by zero or invalid input. Add else to show the result if no error occurs and finally to print “Program finished." Use raise to stop the program if the number is negative, and assert to check the number is positive.

QUESTION SESSION

THANK YOU

https://gobeshona.com.bd/